SoK: Authorization for Autonomous Agents

Anonymous Author(s)

ABSTRACT

An autonomous agent calls tools and Model Context Protocol (MCP) servers while carrying a bearer token (an OAuth 2.0 access token or an MCP authorization token) on each request. Because the agent is driven by untrusted input, its authorization layer is part of the attack surface: a token not bound to the server it is sent to can be replayed or passed through, an over-broad scope hands an injected agent far more than any task needs, and an on-behalf-of token with no verifiable delegation chain leaves "who authorized this?" unanswerable. A fast-growing research literature studies agent security, but it concentrates on model-level attacks (prompt injection, tool poisoning) and proposes individual mechanisms; it does not systematize the authorization layer. We survey 44 research works and show that two authorization properties, audience binding and confused-deputy / dynamic client registration, are treated as mere specification obligations rather than as properties anyone tests or enforces. We then systematize agent authorization along three axes no prior work combines: (i) by *pre-deployment testability*, meaning which threats are decidable from a token, a key set, and a declared tool contract before the agent runs; (ii) by *normative provenance*, a threat-to-clause mapping to the OAuth RFCs, the MCP authorization specification, OWASP, and CWE; and (iii) by the *static-versus-runtime enforcement line*. Across fifteen threats in six categories, we find six are statically contract-verifiable and nine are irreducibly runtime, and we argue this line should guide both tooling and standardization.

1 INTRODUCTION

When an agent calls a tool or an MCP server on a user's behalf, a bearer token travels with the request. The failure modes are OAuth failure modes, but with sharper consequences, because the caller is autonomous and can be steered by untrusted input such as a retrieved document or a tool description [**?**]. A token not bound to the resource it is presented to can be replayed against, or forwarded through, a different server (a confused deputy); an on-behalf-of (OBO) token with no verifiable delegation chain makes accountability impossible; an omnibus scope such as `admin:*` hands an injected agent the keys to everything; and a non-expiring credential never stops being useful to whoever obtains it.

A large and fast-moving body of research now studies agent and MCP security. As our survey (§5) shows, it is dense on *model-level* attacks (prompt-injection-driven escalation and tool poisoning) and it proposes many *individual* defense mechanisms (privilege controllers, delegation protocols, identity architectures). What it does not do is systematize the *authorization* layer itself: no surveyed work sorts agent-authorization threats by whether a defender can test them before deployment, maps each threat to the specific normative clause it violates, or names the line between what can be enforced statically at the trust boundary and what can only be enforced at runtime. The two authorization properties that matter most for token safety, audience binding and the confused-deputy / dynamic-client-registration pair, are treated across 44 works as specification obligations to cite rather than as properties to verify (§5).

Contributions. (i) We give a source-grounded threat model of fifteen agent-authorization threats in six categories, each with its broken invariant, the conditions under which it is exploitable, a mitigation, and its normative source (§2, §3). (ii) We systematize this space along three axes no prior work combines: testability, normative provenance, and the static/runtime enforcement line (§4). (iii) We survey 44 research works on agent authorization and make the coverage gap precise: audience binding and confused-deputy are the under-formalized seam (§5). (iv) We argue which authorization properties a standard or a pre-deployment tool could mandate, and release an open-source analyzer for the statically decidable subset (§4).

2 BACKGROUND

Tokens and claims. An OAuth 2.0 access token authorizes a client to call a resource on a user's behalf; in agent deployments it is usually a JSON Web Token (JWT), a signed set of claims. The claims that govern authorization are the issuer (`iss`), the intended audience (`aud`, the resource the token is *for*), the granted `scope`, the expiry (`exp`), and, for delegation, the authorized party (`azp`, naming the client) and the actor (`act`, naming who is acting). A verifier must check the signature against the issuer's keys, that `aud` names itself, that the algorithm is on an allow-list, and that required claims hold (RFC 8725). RFC 9068 profiles JWT access tokens and makes audience and issuer validation mandatory for that profile; RFC 8707 (resource indicators) lets a client request a token bound to a specific resource.

Delegation and the `act`/`azp` claims. OAuth token exchange (RFC 8693) issues a delegated token carrying an `act` claim that names the acting party and a `may_act` claim stating who is *permitted* to act for the subject. A single `act` claim records one hop; a chain (user → agent → sub-agent) requires *nested* `act` claims, and transitive delegation is only verifiable if each exchange enforces `may_act` and the verifier walks the full nested chain. `azp` names the client that obtained the token but does not by itself establish a delegation chain: it identifies the immediate party, not the sequence of authorizations behind it. This is why `azp`/`act` alone are necessary but not sufficient: without enforced `may_act` and nested-chain validation, "who authorized this?" remains unanswerable across more than one hop.

MCP, identity, and the confused deputy. The Model Context Protocol standardizes how an agent discovers and calls tools; its authorization specification inherits OAuth and *prohibits* passing a client's token through to an upstream API. Dynamic client registration (DCR, RFC 7591) lets clients register at runtime, which an MCP proxy may expose. Workload-identity substrates (WIMSE;

Conference'17, July 2017, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

SPIFFE/SPIRE) would give each agent instance a verifiable, short-lived identity distinct from the human's, but are rarely deployed. The confused-deputy problem (CWE-441), where a more privileged party misuses its authority on behalf of a less privileged one, is the classical frame for token passthrough and consent-flow abuse.

The agent trust model. Two properties of the agent setting sharpen every OAuth failure above. First, the caller is *autonomous*: it chooses which tool to call and with which arguments, so a credential it holds is exercised without a human reviewing each use. Second, its inputs are *untrusted*: an agent reads documents, web pages, and tool descriptions an attacker may control, so prompt injection can steer it into requests the user never intended [**?**]. Together these mean the agent must be treated as a party that can be induced to misuse whatever its token permits, the assumption our threat model adopts and that recent multi-agent security agendas make explicit [**?**]. The consequence for authorization is concrete: the token an agent carries, not a human's judgement at call time, is the last line of defense, so the properties that token enforces are what bound the blast radius of an injected agent. This is why we treat the token-level, statically checkable properties as first-class rather than as spec footnotes.

3 A THREAT MODEL FOR AGENT AUTHORIZATION

We organize the space into six categories. Table 1 lists fifteen threats with the invariant that must hold, a one-line mitigation, and whether a single-token or server-manifest check can decide the threat before deployment. We then give, per category, the exploit condition and the defense.

Audience binding (MCP-01/02). A bearer token names, in its `aud` claim, the resource it is for. The exploit has two shapes. In *passthrough* (MCP-01), an MCP server that receives a client's token forwards it unchanged to an upstream API; because the upstream often shares an issuer it accepts the token, and the MCP server has acted as a confused deputy wielding the full authority of the user's token. In *replay* (MCP-02), a token minted with no resource indicator, or one whose `aud` a verifier never checks, is accepted at any server the issuer serves, so an attacker who captures one token reaches many. Both are properties of claims the token carries and of the server's declared identity, so a defender mints the adversarial token and confirms rejection before deployment. The defense is to request resource-bound tokens (RFC 8707), to verify `aud` names this server, and to honor the MCP prohibition on passthrough. Among the works we survey only Agentic JWT [**?**] and one enterprise framework [**?**] treat audience binding as a property to enforce rather than a specification clause to cite (§5).

Delegation (DEL-01..04). When an agent acts for a user, an on-behalf-of (OBO) token should carry an `act` claim naming the acting party (RFC 8693). A single `act` records one hop; a chain user → agent → sub-agent needs *nested* `act` claims, and the chain is trustworthy only if every exchange along it enforced the `may_act` claim that states who may act for the subject. The exploit: with a missing or forged `act` (DEL-01) an action cannot be attributed, and with an unverified chain (DEL-02) a sub-agent silently exceeds what was delegated, a transitive-trust escalation. Two of the four are statically checkable: a verifier confirms an `act` claim is present and the nested chain is well-formed from the token alone. The other two are runtime: whether the token endpoint *enforced* `may_act` on each exchange (DEL-03), and whether a delegated rather than an impersonated token was issued (DEL-04), are properties of the issuing flow, not of the presented token. Delegation protocols that bind each action to user intent and bound a sub-agent's authority target this category [**? ? ? ?**].

Scope (SCOPE-01..03). The `scope` and `exp` a token carries bound what an injected agent can do with it. The exploit: an omnibus scope such as `admin:*` (SCOPE-01) hands an injected agent far more than any task needs, and a non-expiring token (SCOPE-02) stays useful to whoever later obtains it. Both are read directly from the token, so a check against a declared allow-list and a maximum lifetime is a pre-deployment test. The third, an agent reusing the human's root credential (SCOPE-03), is a property of how the deployment provisions credentials, not of any single token, and so is runtime. Least-privilege frameworks enforce the scope constraint at runtime with monotonic confinement [**?**], and measurements show over-privilege is common in practice [**?**]; our point is that the carried scope and lifetime are checkable before the agent ever runs.

Agent identity (AID-01/02). An agent needs a verifiable identity distinct from the human's so its actions can be attributed and its access revoked. The exploit: treating a session token as authentication (AID-01), or giving the agent no workload identity at all (AID-02), leaves no principal to revoke or audit. The defense is a deployed workload-identity substrate (WIMSE, SPIFFE/SPIRE) that issues each agent instance a short-lived, verifiable credential [**? ? ?**]. Both are runtime: with no identity layer deployed there is nothing in a presented token to check against, which is why they sit on the runtime side of the line.

Confused deputy (CD-01/02). An OAuth proxy sits between many clients and a resource. The exploit: if the proxy reuses one consent across clients, or matches `redirect_uri` loosely, one client inherits another's authorization (CD-01); and if it exposes dynamic client registration without vetting (RFC 7591), an attacker registers a client the proxy then trusts (CD-02). These are properties of a live consent flow and of how the proxy matches redirects at request time, so they cannot be decided from a token or a static manifest. Protocol-level analyses of MCP surface exactly these trust-propagation gaps [**? ?**], yet across the literature the confused deputy appears almost only as a named taxonomy entry, not an enforced property (§5).

Agency (AGT-01/02). The agency threats are where authorization meets the model. The exploit: untrusted input drives the agent to escalate privilege (AGT-01) or to call a tool whose description has been poisoned (AGT-02), so the agent itself issues the malicious request with its own legitimate token. No token check helps, because the token is valid and the request is the agent's own; the decision depends on the executing context. The defenses are runtime: per-action authorization or a human in the loop so an injection cannot escalate on its own [**?**], and vetting of tool descriptions before they reach the model [**? ?**]. This category is the most densely studied in the literature (§5), which is part of our point: the field has concentrated on the model-level attacks and under-formalized the token-level guarantees.

Table 1: Agent-authorization threats by category, with the invariant that must hold, a mitigation, the normative source, and whether the threat is *statically* decidable from a single token or server manifest before deployment (√) or is irreducibly runtime (×). Sev.: severity.

ID	Sev.	Broken invariant (exploit)	Mitigation	Source	Static
Audience binding					
MCP-01	crit	token accepted with wrong/any audience (passthrough, replay)	verify `aud` names this resource; reject forwarded tokens	RFC 8707/9728; MCP [? ? ?]	√
MCP-02	high	token carries no resource indicator (unbound audience)	request resource-bound tokens; require `aud`	RFC 8707 [?]	√
Delegation					
DEL-01	high	missing or forged `act` claim (no accountable actor)	require a signed `act` claim on OBO tokens	RFC 8693 [?]	√
DEL-02	high	unverifiable nested delegation chain (transitive-trust abuse)	validate the full nested `act` chain	RFC 8693 [?]	√
DEL-03	high	`may_act` not enforced on exchange	enforce `may_act` at the token endpoint	RFC 8693 [?]	×
DEL-04	med	impersonation where delegation was required	issue delegated (not impersonated) tokens	RFC 8693 [?]	×
Scope					
SCOPE-01	high	over-scoped agent credential (`admin:*`)	least-privilege scopes per task	RFC 9700; OWASP [? ?]	√
SCOPE-02	high	non-expiring / long-lived token	short `exp`; rotation	RFC 8725 [?]	√
SCOPE-03	high	agent reuses the human's root credential	distinct agent credential	OWASP [?]	×
Agent identity					
AID-01	high	session used as authentication	bind to a verifiable workload identity	WIMSE/SPIFFE [? ?]	×
AID-02	med	no distinct workload/agent identity	issue per-agent SVID/credential	WIMSE/SPIFFE [? ?]	×
Confused deputy					
CD-01	crit	proxy acts on a client's behalf without per-client consent	per-client consent; exact `redirect_uri` match	CWE-441; MCP [? ?]	×
CD-02	high	dynamic client registration abuse	vet/limit DCR; bind registrations	RFC 7591 [?]	×
Agency					
AGT-01	crit	prompt-injection-driven privilege escalation	per-action authorization; human-in-loop	OWASP [? ?]	×
AGT-02	high	tool poisoning / rug-pull via tool metadata	vet tool descriptions; pin versions	OWASP [?]	×

4 SYSTEMATIZING THE SPACE

Our systematization rests on three axes that, together, no prior work applies to agent authorization (§5).

(1) Pre-deployment testability. A threat is *statically decidable* when a defender can reproduce it from a single token, the issuer's key set, and the server's declared tool contract, *without running the agent*. By this criterion six of the fifteen threats are statically decidable: audience binding (MCP-01/02), the `act`-chain delegation checks (DEL-01/02), and scope and expiry (SCOPE-01/02). Each is a property of claims a token carries or of a server's declared identity, so a defender can mint the adversarial token an attacker would and confirm the verifier rejects it, as a check that runs on every build. The remaining nine depend on a live consent flow (CD-01/02), server-side enforcement or a human-in-the-loop (DEL-03/04, SCOPE-03, AGT-01/02), or a deployed workload-identity substrate (AID-01/02), and cannot be settled from a token.

(2) Normative provenance. Each threat maps to the clause it violates: audience binding to RFC 8707 and the MCP passthrough prohibition; delegation to RFC 8693; scope and expiry to RFC 9700 and RFC 8725; registration to RFC 7591; confused deputy to CWE-441; agency to the OWASP LLM and Agentic Top-10s [? ?]. This lets a practitioner trace a defense to the authority that mandates it, a mapping the surveyed literature does not provide.

(3) The static/runtime enforcement line. The defenses in the literature scatter across a line they never name: *static* enforcement at the trust boundary (control/data-flow extraction [?], capability attestation, contract checks on tokens and key sets) versus *runtime* enforcement during execution (monotonic privilege confinement [?], pre-action authorization [?], mandatory access control [?], out-of-band policy [?]). Drawing the line explicitly reframes several "runtime-only" defenses as compensating controls for threats a pre-deployment contract check could have excluded: if audience binding and scope are verified statically, a runtime controller has strictly less to contain. Figure 1 places the fifteen threats on this line.

An instrument for the static subset. We release an open-source analyzer that decodes a presented token and reports cited findings for the six statically decidable threats (audience binding, `act`-chain delegation, scope, and expiry) against a policy. It is stateless, makes no network calls, and runs in CI or beside the agent, so a team red-teams its own authorization before shipping. The taxonomy names the other nine as out of static scope, which is itself the result: it says what a pre-deployment tool, or a tightened standard, can and cannot be expected to guarantee.

5 RELATED WORK

We surveyed 44 research works on authorization, delegation, identity, and token security for agents and MCP (2023–2026); all arXiv identifiers were verified to exist. They divide into *threat taxonomies* and *point mechanisms*, and neither systematizes authorization by our three axes.



Figure 1: The static/runtime enforcement line. Six of the fifteen threats are contract-verifiable before deployment; the other nine are irreducibly runtime. The line is the paper's organizing result: it says what a pre-deployment tool, or a tightened standard, can and cannot be expected to guarantee.

Taxonomies and surveys. MCP security surveys enumerate attack surfaces across the server lifecycle [?], and recent systematizations consolidate attacks and defenses for the ecosystem [?] and for multi-agent systems end to end [?]. Threat taxonomies name authorization-adjacent risks explicitly, including the confused deputy and over-broad consent [?], and a defense-placement taxonomy classifies where in the architecture a control sits [?]. Protocol-level analyses dissect the MCP specification itself, finding missing capability attestation and implicit trust propagation across chained servers [? ?]. These organize by attacker type, lifecycle phase, defense placement, or ecosystem component, and none sorts threats by pre-deployment testability or binds each to the clause it violates.

Delegation and identity mechanisms. A line of work makes delegation verifiable: Agentic JWT binds each agent action to a user intent in the token [?]; AIP carries delegation chains across MCP and A2A boundaries [?]; bounded-delegation and stateful-effect schemes keep a sub-agent within what was delegated [? ?]; and secret-use delegation and residual authorization treat scoped secret handling and revocation [? ?]. Identity work gives agents a verifiable, distinct principal through WIMSE and SPIFFE/SPIRE [? ? ?], and full governance architectures combine registration, identity, and access tokens [?]. These are the mechanisms our delegation and identity categories draw on, each addressing a slice rather than the authorization layer as a whole.

Privilege control and model-level defenses. Runtime privilege frameworks enforce least privilege with symbolic rules and monotonic confinement [?], move the authorization decision before each tool call [?], apply mandatory access control to agent tool use [?], isolate tools in separate environments [?], and enforce policy out-of-band at a trusted boundary [?]; measurements show over-privilege is widespread in practice [?]. For the model-level threats, by-design defenses extract control and data flow so untrusted input cannot alter program flow [?], benchmarks quantify injection success [?], and tool-poisoning work measures and defends the description surface [? ? ? ?], with indirect prompt injection the seminal threat [?]. Each is one point on the static/runtime line; none draws the line, and the densest coverage is on the model-level attacks rather than the token-level guarantees.

The coverage gap. Scoring every work on seven threat categories (Table 2) makes the gap visible: two columns are thin. Audience binding is a central contribution in essentially one research work (Agentic JWT [?]) plus one enterprise framework [?], and confused-deputy / registration appears only as a named taxonomy entry, almost never as a verified or enforced property. Prompt injection and tool poisoning, by contrast, are densely covered. The field has thoroughly characterized the model-level attacks and under-formalized the token- and consent-level authorization guarantees, the seam this SoK occupies.

6 DISCUSSION: WHAT A STANDARD COULD MANDATE

The testability axis has a direct standardization reading. The six statically decidable threats are exactly the ones a profile could make mandatory-to-verify and a conformance test could check: an MCP or OAuth profile that *requires* resource indicators (RFC 8707), rejects passthrough, mandates nested `act` validation, and caps scope and lifetime would move these threats from "deployer's responsibility, usually missed" to "checked at the boundary." The nine runtime threats cannot be closed by a static profile and instead mark where the ecosystem still needs deployed consent, delegation enforcement, and workload identity.

7 THREATS TO VALIDITY

The MCP specification is fast-moving; this systematization is a dated snapshot. The static/runtime line depends on token and manifest availability; a deployment that hides relevant state reduces what any static check can conclude. Our survey covers research indexed by 2026-10; most agent-authorization work is recent and not yet peer-reviewed, which we mark. A prevalence measurement of the statically checkable weaknesses across public MCP deployments is future work, with a hand-validated sample and coordinated disclosure.

ETHICS CONSIDERATIONS

The systematization uses only public specifications, advisories, and CVEs. The accompanying analyzer runs on tokens the operator already holds and sends nothing anywhere; we probe no third-party agents or servers and use no real user tokens. Any future measurement will characterize population prevalence, not single deployments, and will follow coordinated disclosure for identifiable high-severity findings. No personal data or human subjects are involved.

OPEN SCIENCE

We release the static analyzer and a machine-readable threat taxonomy (each threat with its invariant and normative sources) under an open licence, so the systematization is inspectable and the statically checkable subset is reproducible. A DOI-pinned artifact will accompany the camera-ready version.

Table 2: Coverage of the seven agent-authorization threat categories across representative works. √: a central contribution; ○: partially addressed; blank: not addressed. AudBind, audience binding and passthrough; OBO, on-behalf-of delegation; Scope, scope minimization; ConfDep, confused deputy and dynamic registration; WID, workload/agent identity; PI, prompt-injection escalation; Tool, tool poisoning. Two columns, AudBind and ConfDep, are thin.

Work	AudBind	OBO	Scope	ConfDep	WID	PI	Tool
Taxonomies and surveys							
MCP Landscape [?]	○	○	○	○	○	√	√
SoK MCP [?]	○	○	○	○	○	√	√
MCP-38 [?]	○	○	○	√	○	√	√
Enterprise MCP [?]	√	√	√	√	○	√	√
Breaking the Protocol [?]	○			○	○	√	√
Delegation and identity mechanisms							
Agentic JWT [?]	√	√	√	○	√	○	
AIP [?]	○	√	○	○	√		
Bounded Agents [?]	○	√	√	○	○	○	
AID-Guard [?]	○	√	√	○	○	○	
SAGA [?]	○	√	√	○	√	○	
AI Identity [?]	○	√	○	○	√		
aiAuthZ [?]	√	○	○	○	√		
Privilege / runtime control							
Progent [?]			√			√	○
Pre-Action Authz [?]	○	○	√	○	○	√	
MAC framework [?]		○	√	○		√	○
IsolateGPT [?]			√	○	○	√	○
If Agents Were Angels [?]	○	○	√	○	○	√	○
Model-level attacks and defenses							
Indirect PI [?]						√	○
CaMeL [?]			√	○		√	○
MCPTox [?]			○			√	√
No-Box MCP [?]						√	√

8 CONCLUSION

Agent authorization is a high-stakes surface the research literature has studied around rather than through. We systematize it by pre-deployment testability, by normative provenance, and by the static-versus-runtime enforcement line; we show that six of fifteen threats are contract-verifiable before an agent runs and nine are irreducibly runtime; and we argue this line should guide what a pre-deployment tool checks and what a standard makes mandatory to verify.

LLM USAGE STATEMENT

LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the authors to ensure accuracy and originality. An LLM-assisted process additionally supported the literature survey of §5: candidate works were gathered with model assistance and each was then verified by the authors to exist, with titles and author lists confirmed against the arXiv API on 2026-10-06; works that could not be verified were excluded, and one paper that its own authors had formally withdrawn was identified this way and removed. The authors are responsible for the selection of works, the coverage judgments in the systematization, and every claim in the paper. One reproducibility limitation applies: the assisting models are not openly available, so the survey's search recall cannot be reproduced exactly, although every cited work is independently verifiable by its arXiv identifier.